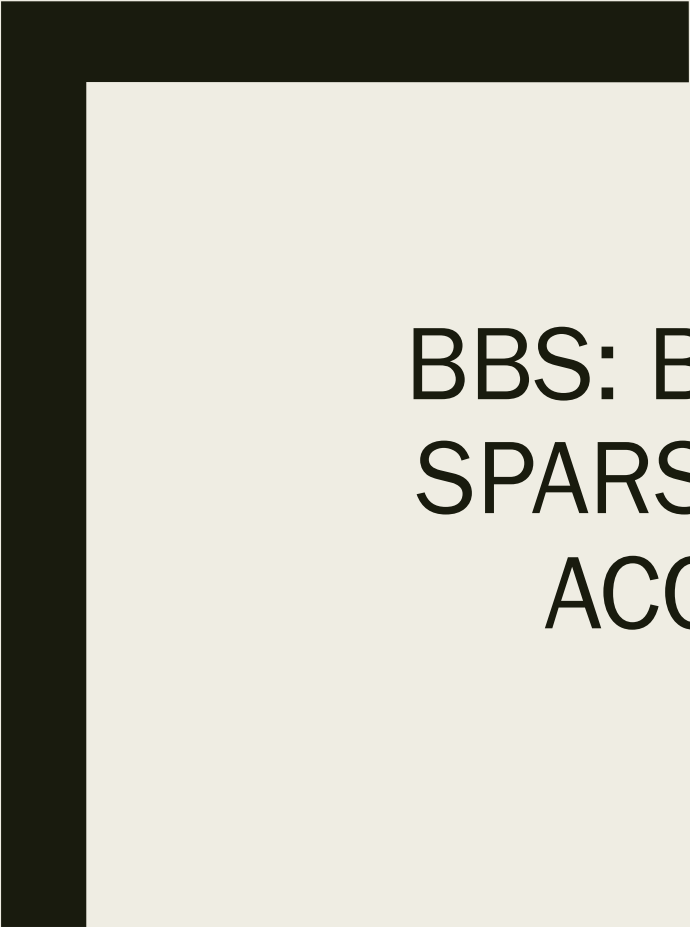
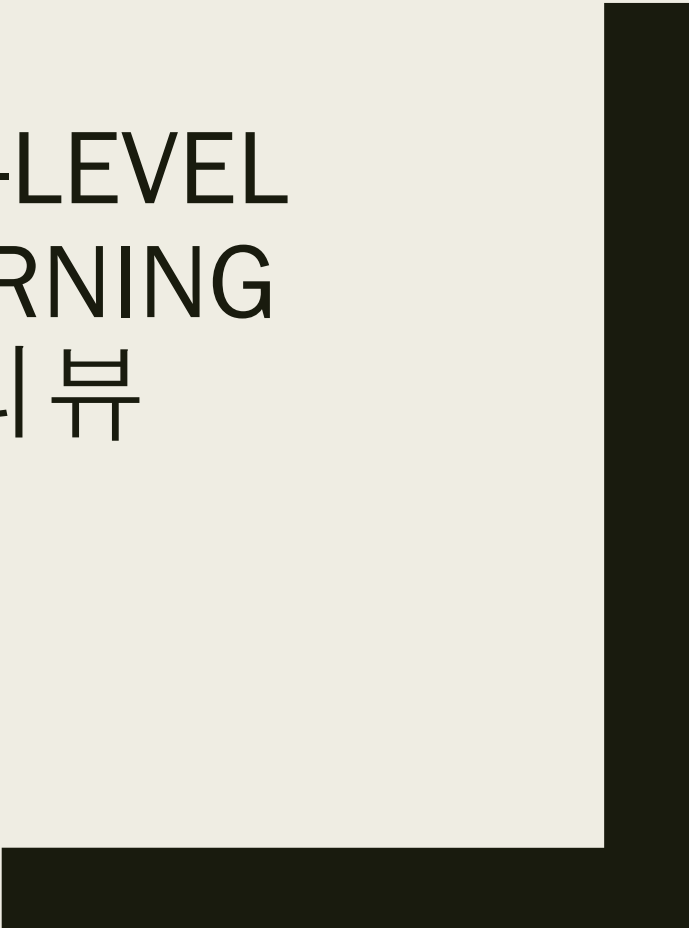




BBS: BI-DIRECTIONAL BIT-LEVEL SPARSITY FOR DEEP LEARNING ACCELERATION 논문 리뷰

26.02.27.세미나



목차

- 1. Abstract
- 2. Introduction
- 3. Background
- 4. BBS: Bi-Directional Bit-Level Sparsity
- 5. Bitvert Hardware Architecture
- 6. Evaluation/Conclusion

Abstract

Abstract

- 비트 수준 희소성(Bit-level sparsity) 방법: 효과가 없는 0비트 연산을 건너뛰는 연산 방식
-> 일반적으로 비트 직렬 딥러닝 가속기 내에서 적용 가능-> 여러 이유로 실용성 부족
- 실용성 부족한 이유
 - 1) 부하 불균형(0비트의 무작위 분포)
 - 2) 모든 비트가 오프칩 메모리에서 호출 -> 최적화되지 않은 외부 메모리 액세스
 - 3) 희소성을 지원하기 위한 대형 멀티플렉서와 같은 높은 하드웨어 구현 오버헤드

Abstract

- 본 연구에서 새로운 방식 제안
- 1) 알고리즘 측면 -> “양방향 비트 희소성”(BBS)
- 2) 하드웨어 측면 -> Bivert
- 실험 결과: 모델 크기 평균 1.66배 감소, 이전 DNN 가속기 대비 최대 3.03배의 속도 향상 + 2.44배 에너지 절감

Introduction

Introduction

- DNN은 많고 중요한 분야에서 놀라운 성과를 입증하고 있음
but DNN 모델 크기와 복잡성의 성장은 기존 하드웨어 플랫폼의 컴퓨팅 성능 향상 속도를 앞지름 -> 격차를 줄이기 위해 고성능과 에너지 효율 모두 필요

- 1) 압축된 모델을 효율적으로 배포하기 위한 가속기
- 2) 새로운 DNN 압축 알고리즘

=> 공동 설계

Introduction

- 수많은 효율성 알고리즘 + 하드웨어 프로토타입 제안됨
but 희소성의 정도는 결과적인 하드웨어 성능을 강력하게 제한함
- 이러한 문제는 LLM에서 두드러짐 -> 재학습을 더욱 자원 및 데이터 집약적으로 만듦
- 재학습을 강요하지 않으면서 가속기의 효율성을 향상시켜야 함

Introduction

- 또다른 흐름은 모델 재학습 $x \rightarrow$ 피연산자를 더 낮은 정밀도로 표현하는 사후 학습 양자화(PTQ)에 초점 맞춤(ex: 32비트 $\rightarrow$ 8비트로 줄임)
- 그러나 이는 정수 양자화보다 더 높은 하드웨어 비용 초래
- 최신 PTQ 알고리즘은 무시 가능 정확도 손실로 피연산자 정밀도를 8비트 정수 (INT8)로 줄일 수 있음 $\rightarrow$ 희소성 매우 낮음
- 양자화-희소성 간의 긴장 $\rightarrow$ 성능 병목 현상 발생

Introduction

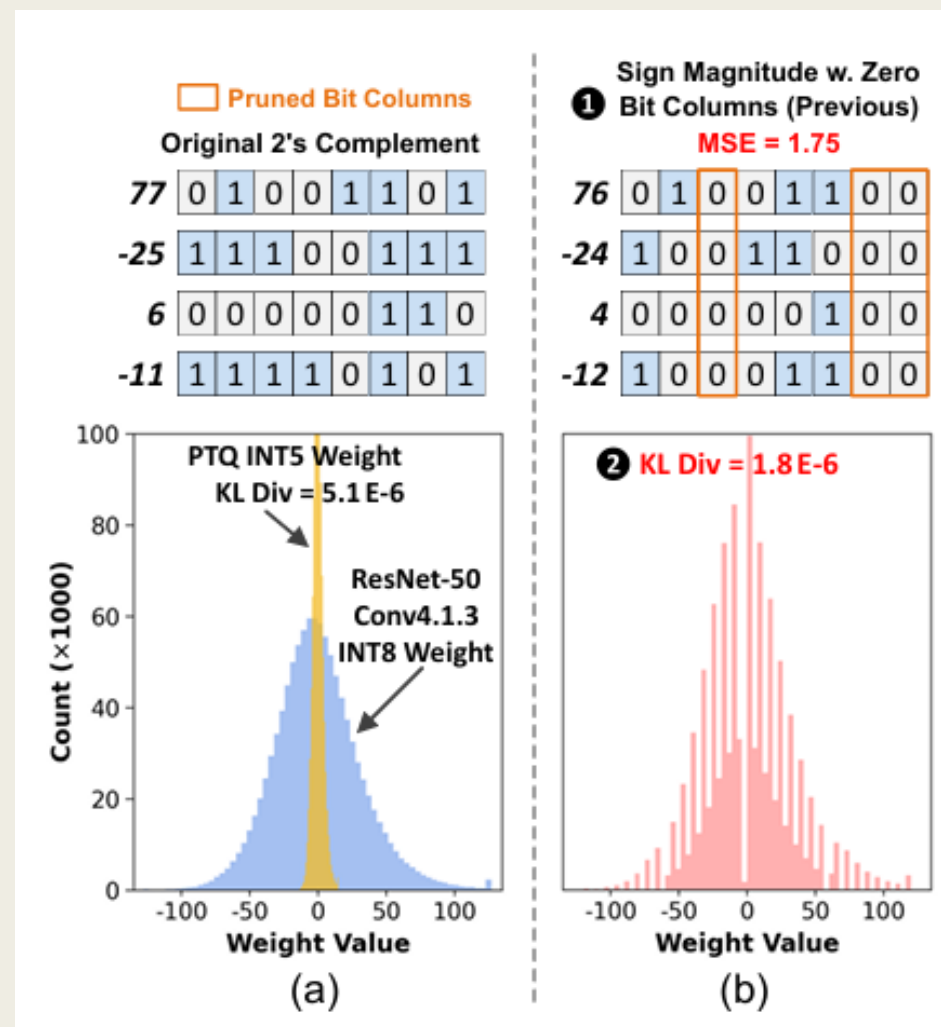
- 양자화와 희소성의 효율성을 동시에 달성하기 위해 비트 수준 희소성 활용
-> 0비트 연산을 건너뛰는 것을 제안함
- But 0비트의 분포는 무작위 -> 상당한 작업 부하 불균형 초래
 - + 모든 데이터 비트를 오프칩 메모리에서 가져와야함
 - + 하드웨어 오버헤드 발생

Introduction

- BitWave는 동일한 비트 중요도와 희소성을 조사하는 비트 열 직렬 방식 채택
- 비트 열이 모두 0비트 -> 메모리 저장 x
+ 부호-크기 형식의 가중치 기반으로 비트 선택적으로 0으로 뒤집는 비트 희소성 강화 기술
- 결과적으로 더 높은 성능 달성 가능성 보여줌
- But 비트 희소성이 “0비트 ”에만 국한됨

Introduction

- (a): 일반적인 PTQ
전체 번위를 32개 칸으로 나눔
큰 값들은 잘리고, 작은 값들은 뭉뚱그래짐
-> 원래 데이터 분포와의 차이 커짐
- (b): 비트 수준 프루닝
비트 열단위로 접근
(ex: 8개중 3개 열 '0'으로 프루닝 -> 남은 비트 숫자로 표현)
숫자가 작을 때 매우 유리 + PTQ보다 정밀하게 원래 값 보존 가능
but 고유한 희소 비트 열x -> 모든 하위 비트 열 0으로 뒤집어야됨 -> 양자화 레벨 감소



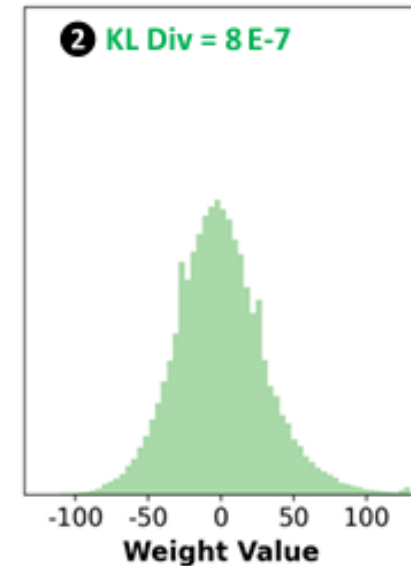
Introduction

- (c): BBS
모든 비트 벡터가 적어도 50%의 BBS를 보일 수 있도록 보장함-
> 부하 균형 개선 + 불필요한 비트 연산 횟수 최소화
- 이 연구의 주요 기여
 - 1) 새로운 BBS 개념 도입 -> 비트 직렬 가속기의 부하 균형 개선
 - 2) 두 가지 비트 수준 바이너리 프루닝 전략 제안
 - 3) BBS 활용 비트 직렬 가속기인 BitVert 설계

2's Complement w.
① Bi-directional Sparse
Columns (Ours), MSE = 0.75

78	0	1	0	0	1	1	1	0
-26	1	1	1	0	0	1	1	0
6	0	0	0	0	0	1	1	0
-10	1	1	1	1	0	1	1	0

② KL Div = 8 E-7

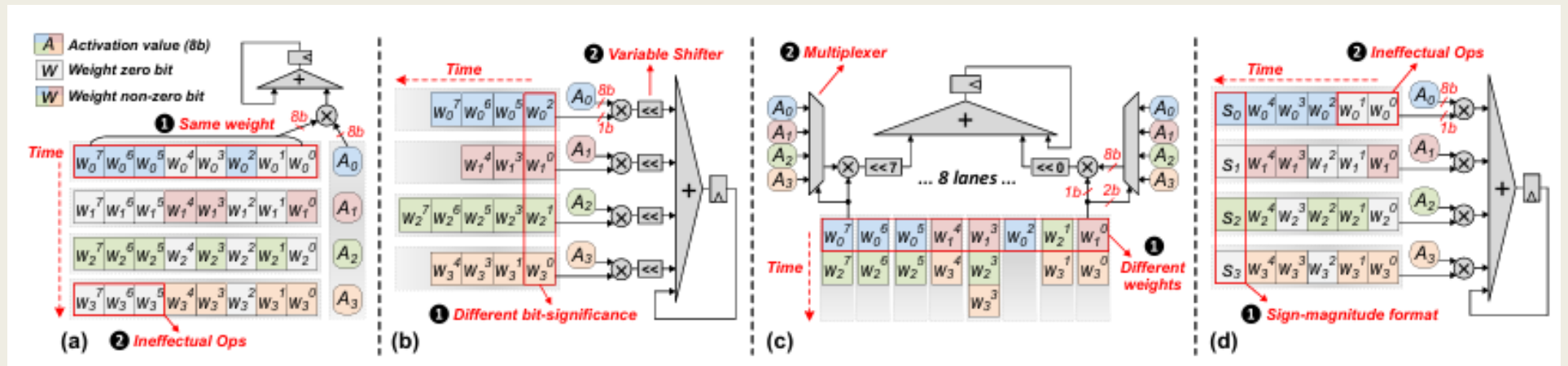


(c)

Background

Background

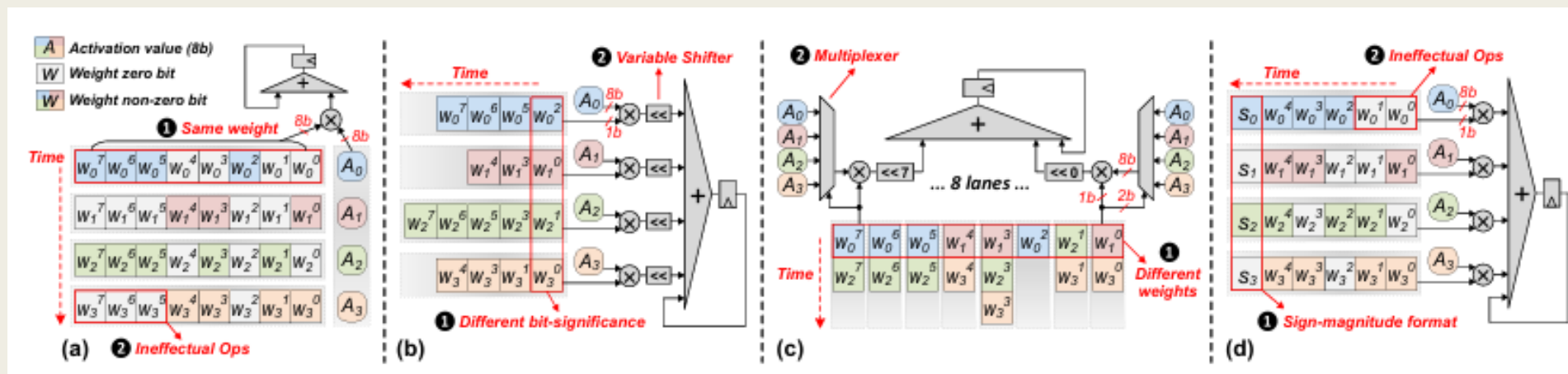
- (a): 비트 병렬 PE: 동일한 가중치의 모든 비트 간의 곱셈 수행 -> 의미 없는 비트 연산 초래 -> 성능 저하
- (b): Pragmatic: 모든 가중치의 비트 중 0이 아닌 비트만 처리
서로 다른 비트 가중치가 동시 처리될 수 있음
-> 가중치 동기화 위해 모든 비트 직렬 곱셈기 뒤에 가변 시프터 필요
- Pragmatic은 부하 불균형 문제 발생(지연 시간이 1비트 개수가 가장 많은 가중치에 의해 결정됨)



Background

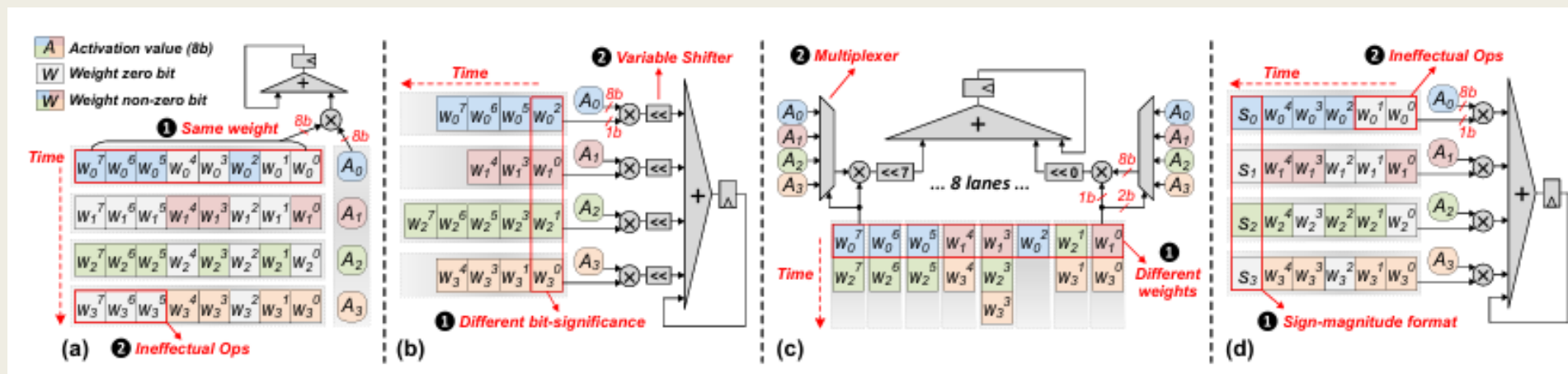
- (c): Bitlet: 여러 가중치와 활성화 소화 -> 각 비트 가중치 독립적으로 계산

but 모든 비틀 lane이 임의의 가중치로부터 유효 비트 흡수할 수 있음 -> 올바른 활성화 선택을 위한 대형 멀티플렉서 필요 -> 하드웨어 오버헤드 초래
+ 부하 불균형 문제



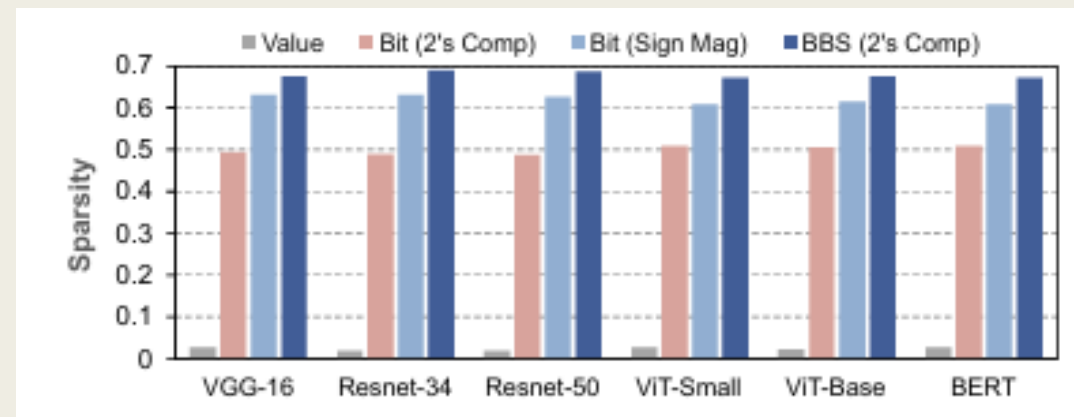
Background

- (d): BitWave: 열 단위로 0비트를 건너뛴
- BirVert: 가능한 많은 희소 비트 건너뛴 + 비트 직렬 작업 부하의 균형 맞추기 위해 노력
- If 비트 열에 0이 많음 -> 0비트 건너뛴
- If 비트 열에 0이 적음 -> 1비트를 건너뛴



Background

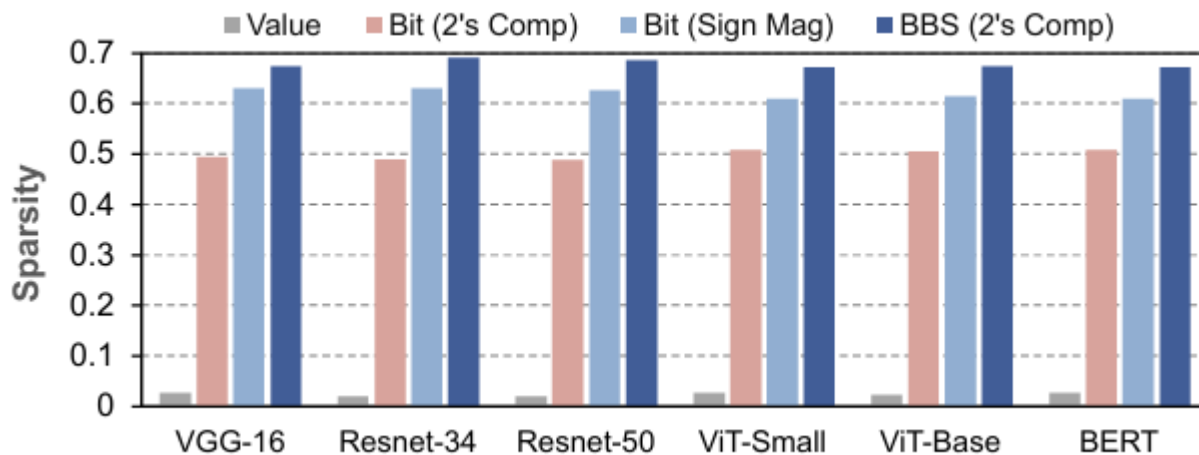
- 대중적인 8비트 양자화 DNN에서 값 기반 가중치 희소성은 5% 미만
- 비트 수준 희소성은 50% 도달할 수 있음
- 비트 직렬 컴퓨팅은 부호-크기 산술 채택에 여전히 문제 있음
 - 1) 모든 비트 직렬 곱셈기는 부분합 생성을 위해 2의 보수 변환기 필요
 - 2) 부하 불균형과 동기화 오버헤드 초래
- BBS는 2의 보수 이진 표현 유지 + 발생 빈도가 더 높은 0 또는 1을 희소 비트로 취급
-> 모든 비트 벡터가 적어도 50% 비트 희소성 보장+ 서로 다른 PE 간에 균형 잡힌 작업 부하



BBS: Bi- Directional Bit- Level Sparsity

BBS

- N: 그룹 크기(그룹: 동일 내적 출력에 기여하는 여러 가중치 또는 활성화)
- 가중치의 모든 비트는 0 or 1
- If 비트 벡터에 50% 이상의 0비트: 연산 중에 건너뛸
- Else: 비트 벡터 반전시킴 -> 전체 합에서 해당 비트 직렬 내적 값 뺌
- 이 BBS 아이디어는 부하 균형을 개선할 수 있음

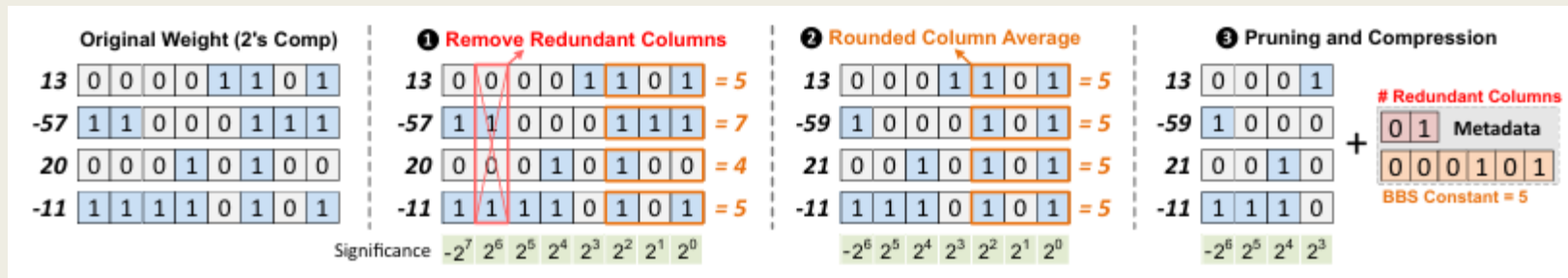


$$\sum_{i=0}^{N-1} W_i \times A_i = \sum_{b=0}^{p-1} 2^b \times \sum_{i=0}^{N-1} W_i^b \times A_i$$

$$\begin{aligned} \sum_{i=0}^{N-1} W_i^b \times A_i &= \sum_{\forall i: W_i^b=1} A_i \\ &= \sum_{j=0}^{N-1} A_j - \sum_{\forall i: W_i^b=0} A_i \end{aligned}$$

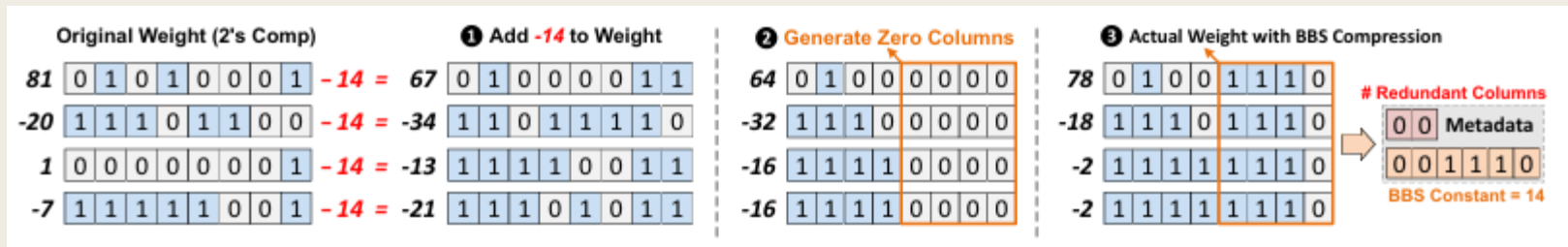
BBS

- 비트 수준 바이너리 프루닝: 가중치 그룹 내에서 모두 0비트이거나 모두 1비트인 비트 열 제거하는 기술
- 반올림 평균화 기반 BBS
 - 1) 최상위 비트 열과 내용이 동일하며 바로 뒤따라오는 중복 비트 열 확인
 - 2) 원래 가중치의 하위 3개 비트가 나타내는 값들의 반올림 평균을 계산
 - 3) 남은 4개의 비트 열과 8비트 인코딩 메타데이터 저장하여 원래 가중치 그룹 압축



BBS

- 제로 포인트 시프팅 기반 BBS
- 1) 상수를 더한다고 가정 => 모든 숫자의 이진 내용 변경
- 2) 4개의 하위 비트 열 프루닝(제거)할때 숫자의 하위 4비트 직접 0으로 만들 or 상위 비트로 반올림
- 3) 바이너리 프루닝 후의 실제 값 보여줌 -> 새로운 제로 포인트 인코딩 메타데이터에 저장



BBS

Algorithm 1: Finding the optimal constant for zero-point shifting.

Input : Weight group: W , BBS constant precision: p ,
target number of sparse bit columns: N

Output : Compressed weight: W_C , metadata: D

```
1 def Compress( $W, N, p$ ):
2     bestMSE =  $\infty$  ;
3     for constant =  $-2^{p-1}$  to  $2^{p-1} - 1$  do
4          $W_{tmp} = \text{Clip}(W + \text{constant})$ 
5         // Get number of redundant columns
6         numRedunCol = GetNumRedunCol( $W_{tmp}$ )
7          $W_{tmp} = \text{RemoveRedunCol}(W_{tmp}, \text{numRedunCol})$ 
8         // Generate zero sparse columns
9         numSparseCol =  $N - \text{numRedunCol}$ 
10         $W_{tmp} = \text{GenSparseCol}(W_{tmp}, \text{numSparseCol})$ 
11        newMSE =  $|W_{tmp} - W|^2$ 
12        if newMSE < bestMSE then
13            bestMSE = newMSE
14             $W_C = W_{tmp}$ 
15             $D = \{ \text{numRedunCol}, \text{constant} \}$ 
16    return  $W_C, D$ 
```

Bitvert Hardware Architecture

Evaluation/Conclusion

Evaluation

- 보통 모델 양자화 시 실제 이미지 데이터 필요
but BBS는 데이터 실제 이미지 데이터 없이도 정확도 유지
- 속도도 3.03배 향상 + 에너지도 2.44배 절감
- BitVert는 BBS의 이론인 50%를 보장하는 이론이 적용돼서 병렬 처리 효율이 극대화됨
- 또한, 더 적은 오차로 더 높은 효율을 냄

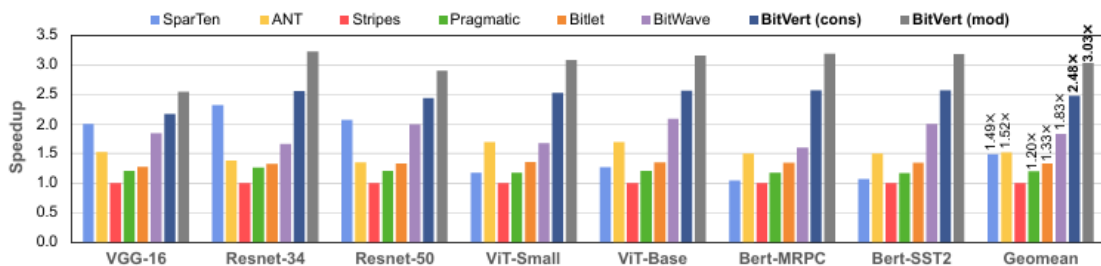
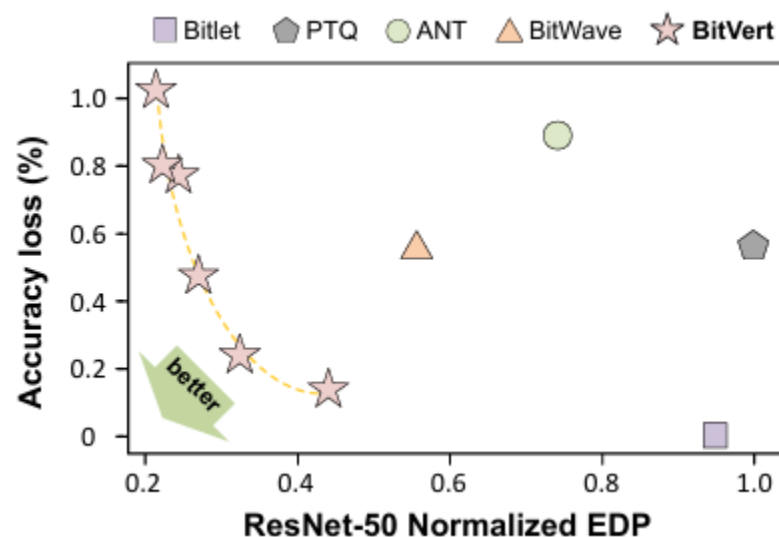
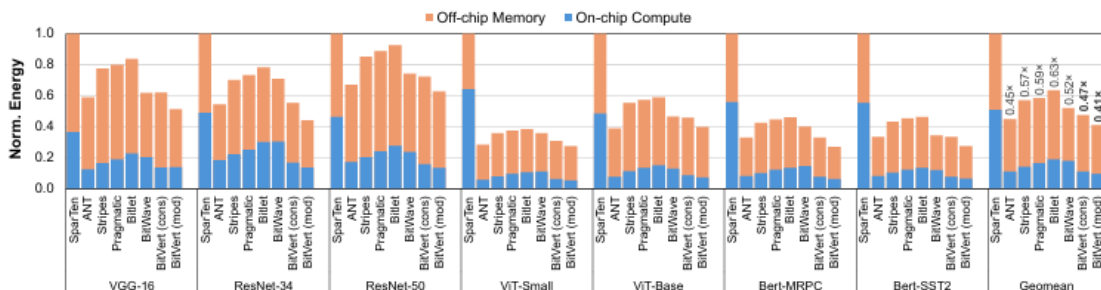


Fig. 12: Speedup results normalized to Stripes (higher is better).



Conclusion

- BBS는 압축되지 않은 모델의 통계적 특성 최대한 보존 + 바이너리 프루닝을 통해 사후 학습 DNN 압축의 한계를 최첨단 수준으로 끌어올림
- 그 결과, 바이너리 프루닝 기술 -> 훨씬 더 높은 정확도 달성
- BitVert 가속기 -> 최대 3.03배의 속도 향상 + 2.44배 에너지 절감